Cache Stampede and Thundering Herd

when one hot key expires and everyone rebuilds it

Watch the 4-min video

In one line: a stampede happens when a hot key expires and thousands of concurrent misses all regenerate it; prevent it with single-flight inside a server, a short distributed lock across servers, early refresh (probabilistic or scheduled) so the key never expires under load, and stale-while-revalidate so users get the old value while one task refreshes it.

1 One key, fifty thousand readers

Your homepage data is cached under one key and read 50,000 times per second. The key expires. In the next 100 milliseconds, thousands of requests miss, and every one of them runs the same expensive query against the database. The database slows, queries pile up, the next requests time out, and the whole site goes down. That's a **cache stampede**.

2 Why it's so dangerous

A stampede, also called **dogpiling** or a **thundering herd**, happens when many requests miss on the same key at the same time and all try to regenerate it. The more popular the key and the slower the regeneration, the worse it gets: 50,000 requests per second × a 200 ms query = about **10,000 concurrent queries for one value**. And the slower the database gets, the longer each query takes, so even more requests pile in.

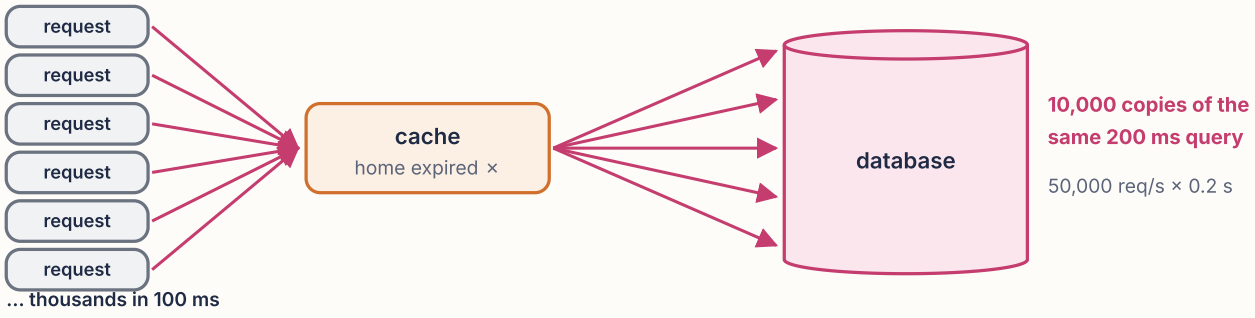


Figure 1. A stampede: every request that arrives while the key is missing runs the same query.

3 Fix 1: request coalescing (single-flight)

Within one server, only the **first** request for a missing key goes to the database. Concurrent requests for the same key wait for that result and share it. Go's `singleflight` package does exactly this; in Python, a per-key `asyncio` future does the same. It turns thousands of queries per server into one per server.

4 Fix 2: a distributed lock

Across many servers, the first request that misses takes a short lock in Redis with `SET lock:home <id> NX PX 5000` (only if not exists, expires in 5 s). It regenerates the value, stores it and releases the lock. Requests that fail to get the lock either wait briefly and re-read the cache, or serve a slightly stale value. Always give the lock an expiry, so a crashed holder doesn't block everyone, and release it only if you still own it (compare the id).

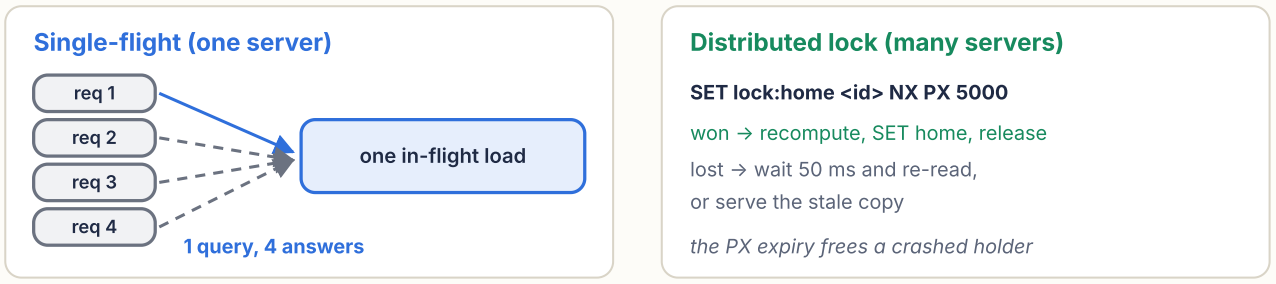


Figure 2. Single-flight collapses misses inside one server; a lock with an expiry lets only one server regenerate.

5 Fix 3: never let a hot key expire under load

Probabilistic early expiration (the **XFetch** algorithm): each request near the end of the TTL has a small, growing chance of refreshing the value early. One lucky request recomputes it before it expires, while everyone else keeps hitting:

```
refresh early if now − delta × beta × ln(random()) ≥ expiry (delta =
                    recompute time, beta ≈ 1)
```

Or simply run a **background job** that refreshes known hot keys on a schedule, so they never expire at all.

6 Fix 4: stale-while-revalidate

Keep serving the old value after it goes stale, while a single background task fetches the fresh one. Users get an instant response, and only one regeneration runs. Implement it with a **soft TTL** stored inside the value (refresh after it) and a longer **hard TTL** on the key (delete after it). HTTP supports the same idea with `Cache-Control: stale-while-revalidate`, and CDNs honour it.

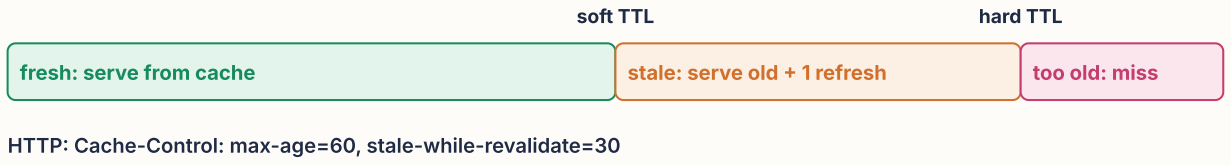


Figure 3. Soft and hard TTLs: between them the cache serves the old value while exactly one request refreshes it.

WORKED EXAMPLE: THE HOMEPAGE KEY

The homepage key is read at 50,000 requests per second, with a 60-second TTL. Without protection, each expiry sends thousands of queries to the database. With a 60-second soft TTL, a 120-second hard TTL, stale-while-revalidate and a Redis lock so only one worker refreshes, each expiry causes **exactly one query**, and users never see a slow page.

COMMON MISTAKES

- A lock without an expiry: one crashed worker blocks the key forever.
- Lock losers that retry immediately in a tight loop, hammering Redis.
- Fixing stampedes on one key but leaving thousands of keys with the same TTL (that's an avalanche, next lesson).

7 Key terms

Cache stampede: many concurrent misses regenerating one key

Dogpiling / thundering herd: other names for the same thing

Single-flight: one in-flight load per key, shared by waiters

Distributed lock: SET NX PX so one server regenerates

XFetch: probabilistic early refresh before expiry

Soft / hard TTL: refresh after soft, delete after hard

Stale-while-revalidate: serve old while one task refreshes

8 Interview questions

Q1. What is a cache stampede? BASIC

When a hot key expires or is evicted, many concurrent requests miss at once and all regenerate it, overloading the database.

Q2. How do you prevent it? BASIC

Coalesce requests with single-flight, use a distributed lock so one request regenerates, refresh early with probabilistic expiration or background jobs, and serve stale values while revalidating.

Q3. Design caching for a homepage read 50,000 times a second. SCENARIO

Cache it in Redis (and in process memory for a second or two), refresh it from a background job or with a soft TTL plus stale-while-revalidate, guard regeneration with a lock, and never let it hard-expire under load.

Q4. Your lock-based fix works, but lock losers add latency. What else? SCENARIO

Let losers serve the stale value instead of waiting, add a short in-process cache in front of Redis, and refresh early so the key is rarely missing at all.

Q5. Why is the lock's expiry important, and what's the risk if it's too short? TRICKY

Without it, a crashed holder blocks the key forever; if it's shorter than the regeneration time, a second worker can acquire it and you get two regenerations. Set it longer than the slowest recompute and check ownership on release.

Q6. How does XFetch avoid stampedes without locks? TRICKY

Each read near expiry refreshes with a probability that rises as expiry approaches (and with longer recompute times), so typically one request refreshes early and the key never expires while popular.

Next: SD-F-011 · Cache Penetration and Avalanche backendroadmap.com